

Homework 2

Joseph Shin

2025-08-25

Question 3.1: Using the same data set (credit_card_data.txt or credit_card_data-headers.txt) as in Question 2.2, use the ksvm or kkn function to find a good classifier: (a) using cross-validation (do this for the k-nearest-neighbors model; SVM is optional); and (b) splitting the data into training, validation, and test data sets (pick either KNN or SVM; the other is optional).

```
library(kernlab)
```

```
## Warning: package 'kernlab' was built under R version 4.3.3
```

```
library(kknn)
```

```
## Warning: package 'kknn' was built under R version 4.3.3
```

```
data = read.csv("credit_card_data-headers copy.txt", header = TRUE, sep = "\t")
```

Question 3.1 a) Cross Validation

```
set.seed(1)
```

```
random = sample(1:nrow(data), as.integer(0.7*nrow(data)))
```

```
training_data = data[random,]
```

```
testing_data = data[-random,]
```

```
train.kknn(R1 ~., data = training_data, kmax = 50, scale = TRUE)
```

```
##
```

```
## Call:
```

```
## train.kknn(formula = R1 ~ ., data = training_data, kmax = 50, scale = TRUE)
```

```
##
```

```
## Type of response variable: continuous
```

```
## minimal mean absolute error: 0.1955315
```

```
## Minimal mean squared error: 0.1085533
```

```
## Best kernel: optimal
```

```
## Best k: 41
```

```

training_prediction = rep(0, (nrow(training_data)))

training_accuracy = 0

for (i in 1:nrow(training_data)) {
  model = kknn(R1 ~., training_data[-i,], training_data[i,], k = 41, kernel = "optimal", scale = TRUE)
  training_prediction[i] = as.integer(fitted(model) + 0.5)
}

training_accuracy = sum(training_prediction == training_data[,11]) / nrow(training_data)

testing_prediction = rep(0, (nrow(testing_data)))
testing_accuracy = 0

for (i in 1:nrow(testing_data)) {
  model = kknn(R1 ~., testing_data[-i,], testing_data[i,], k = 41, kernel = "optimal", scale = TRUE)
  testing_prediction[i] = as.integer(fitted(model) + 0.5)
}

testing_accuracy = sum(testing_prediction == testing_data[,11]) / nrow(testing_data)

training_accuracy

```

```
## [1] 0.833698
```

```
testing_accuracy
```

```
## [1] 0.8375635
```

Question 3.1 b) Splitting into training, validation, and testing data sets

Training data: 60%, Validation data: 20%, Testing data: 20%

```

set.seed(1)

training_index = sample(1:nrow(data), as.integer(0.6 * nrow(data)))
remaining_index = setdiff(1:nrow(data), training_index)
validation_index = sample(remaining_index, as.integer(0.5 * length(remaining_index)))
testing_index = setdiff(remaining_index, validation_index)

training_data = data[training_index, ]
validation_data = data[validation_index, ]
testing_data = data[testing_index, ]

```

```

training_prediction = rep(0, (nrow(training_data)))
training_accuracy = 0
i = 0
accuracytab = data.frame(matrix(nrow = 25, ncol = 2))
colnames(accuracytab) = c("K", "Accuracy")
for (X in 1:25) {
  for (i in 1:nrow(training_data)) {
    model = kknn(R1 ~. , training_data[-i, ], training_data[i, ], k = X, kernel = "optimal", scale = TR

```

```

    training_prediction[i] = as.integer(fitted(model) + 0.5)
  }

  training_accuracy = sum(training_prediction == training_data[,11]) / nrow(training_data)
  accuracytab[X, 1] = X
  accuracytab[X, 2] = training_accuracy
}
accuracytab

```

```

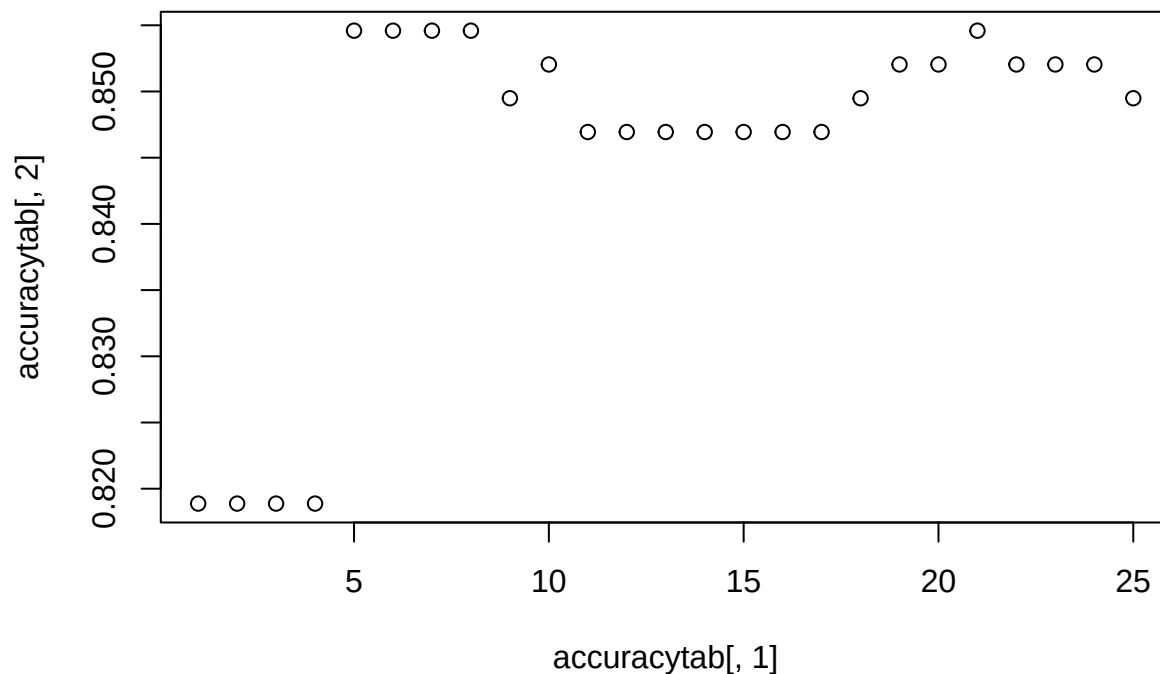
##      K  Accuracy
## 1    1 0.8188776
## 2    2 0.8188776
## 3    3 0.8188776
## 4    4 0.8188776
## 5    5 0.8545918
## 6    6 0.8545918
## 7    7 0.8545918
## 8    8 0.8545918
## 9    9 0.8494898
## 10  10 0.8520408
## 11  11 0.8469388
## 12  12 0.8469388
## 13  13 0.8469388
## 14  14 0.8469388
## 15  15 0.8469388
## 16  16 0.8469388
## 17  17 0.8469388
## 18  18 0.8494898
## 19  19 0.8520408
## 20  20 0.8520408
## 21  21 0.8545918
## 22  22 0.8520408
## 23  23 0.8520408
## 24  24 0.8520408
## 25  25 0.8494898

```

```

plot(accuracytab[,1], accuracytab[,2])

```



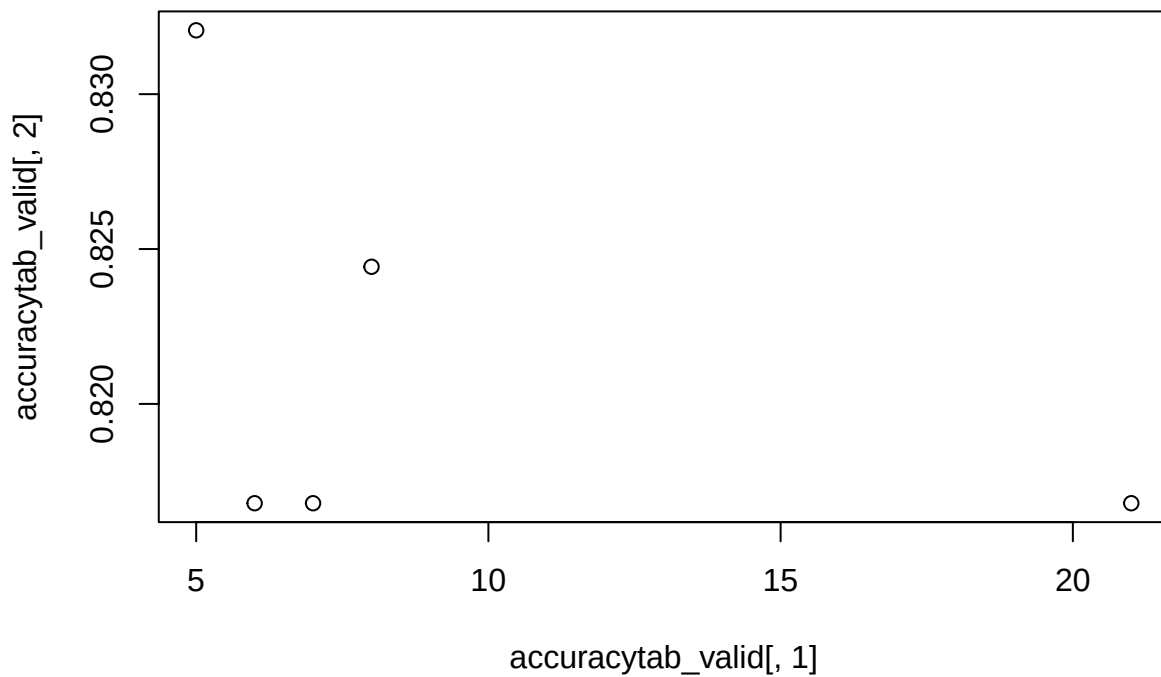
```
validation_prediction = rep(0, (nrow(validation_data)))
validation_accuracy = 0
i = 0
accuracytab_valid = data.frame(matrix(nrow = 25, ncol = 2))
colnames(accuracytab_valid) = c("K", "Accuracy")
for (X in c(5:8, 21)) {
  for (i in 1:nrow(validation_data)) {
    model = kknn(R1 ~. , validation_data[-i, ], validation_data[i, ], k = X, kernel = "optimal", scale = 1)
    validation_prediction[i] = as.integer(fitted(model) + 0.5)
  }

  validation_accuracy = sum(validation_prediction == validation_data[,11]) / nrow(validation_data)
  accuracytab_valid[X, 1] = X
  accuracytab_valid[X, 2] = validation_accuracy
}
accuracytab_valid
```

```
##      K  Accuracy
## 1  NA         NA
## 2  NA         NA
## 3  NA         NA
## 4  NA         NA
## 5   5 0.8320611
## 6   6 0.8167939
## 7   7 0.8167939
## 8   8 0.8244275
## 9  NA         NA
## 10 NA         NA
## 11 NA         NA
## 12 NA         NA
## 13 NA         NA
## 14 NA         NA
```

```
## 15 NA      NA
## 16 NA      NA
## 17 NA      NA
## 18 NA      NA
## 19 NA      NA
## 20 NA      NA
## 21 21 0.8167939
## 22 NA      NA
## 23 NA      NA
## 24 NA      NA
## 25 NA      NA
```

```
plot(accuracytab_valid[,1], accuracytab_valid[,2])
```



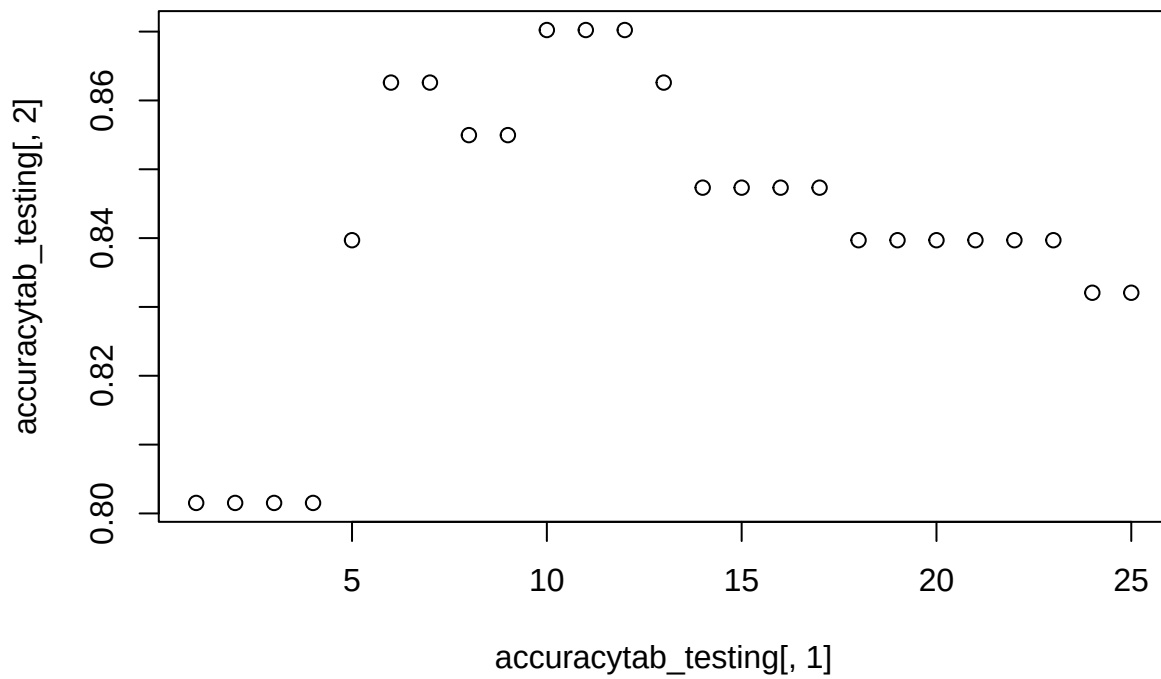
```
testing_prediction = rep(0, (nrow(testing_data)))
testing_accuracy = 0
i = 0
accuracytab_testing = data.frame(matrix(nrow = 25, ncol = 2))
colnames(accuracytab_testing) = c("K", "Accuracy")
for (X in 1:25) {
  for (i in 1:nrow(testing_data)) {
    model = kknn(R1 ~. , testing_data[-i, ], testing_data[i, ], k = X, kernel = "optimal", scale = TRUE)
    testing_prediction[i] = as.integer(fitted(model) + 0.5)
  }

  testing_accuracy = sum(testing_prediction == testing_data[,11]) / nrow(testing_data)
  accuracytab_testing[X, 1] = X
  accuracytab_testing[X, 2] = testing_accuracy
}
accuracytab_testing
```

```
##      K Accuracy
```

```
## 1 1 0.8015267
## 2 2 0.8015267
## 3 3 0.8015267
## 4 4 0.8015267
## 5 5 0.8396947
## 6 6 0.8625954
## 7 7 0.8625954
## 8 8 0.8549618
## 9 9 0.8549618
## 10 10 0.8702290
## 11 11 0.8702290
## 12 12 0.8702290
## 13 13 0.8625954
## 14 14 0.8473282
## 15 15 0.8473282
## 16 16 0.8473282
## 17 17 0.8473282
## 18 18 0.8396947
## 19 19 0.8396947
## 20 20 0.8396947
## 21 21 0.8396947
## 22 22 0.8396947
## 23 23 0.8396947
## 24 24 0.8320611
## 25 25 0.8320611
```

```
plot(accuracytab_testing[,1], accuracytab_testing[,2])
```



Question 4.2: Clustering

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```
##
```

```
## Attaching package: 'ggplot2'
```

```
## The following object is masked from 'package:kernlab':
```

```
##
```

```
##      alpha
```

```
library(tibble)
```

```
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(gridExtra)
```

```
iris = read.table('iris.txt', header = TRUE)
```

```
iris_clean = scale(iris[, -5])
```

```
km2 = kmeans(iris_clean, centers = 2, nstart = 20)
```

```
km3 = kmeans(iris_clean, centers = 3, nstart = 20)
```

```
km4 = kmeans(iris_clean, centers = 4, nstart = 20)
```

```
km5 = kmeans(iris_clean, centers = 5, nstart = 20)
```

```
plot2 = fviz_cluster(km2, geom = "point", data = iris_clean) + ggtitle("Two Clusters")
```

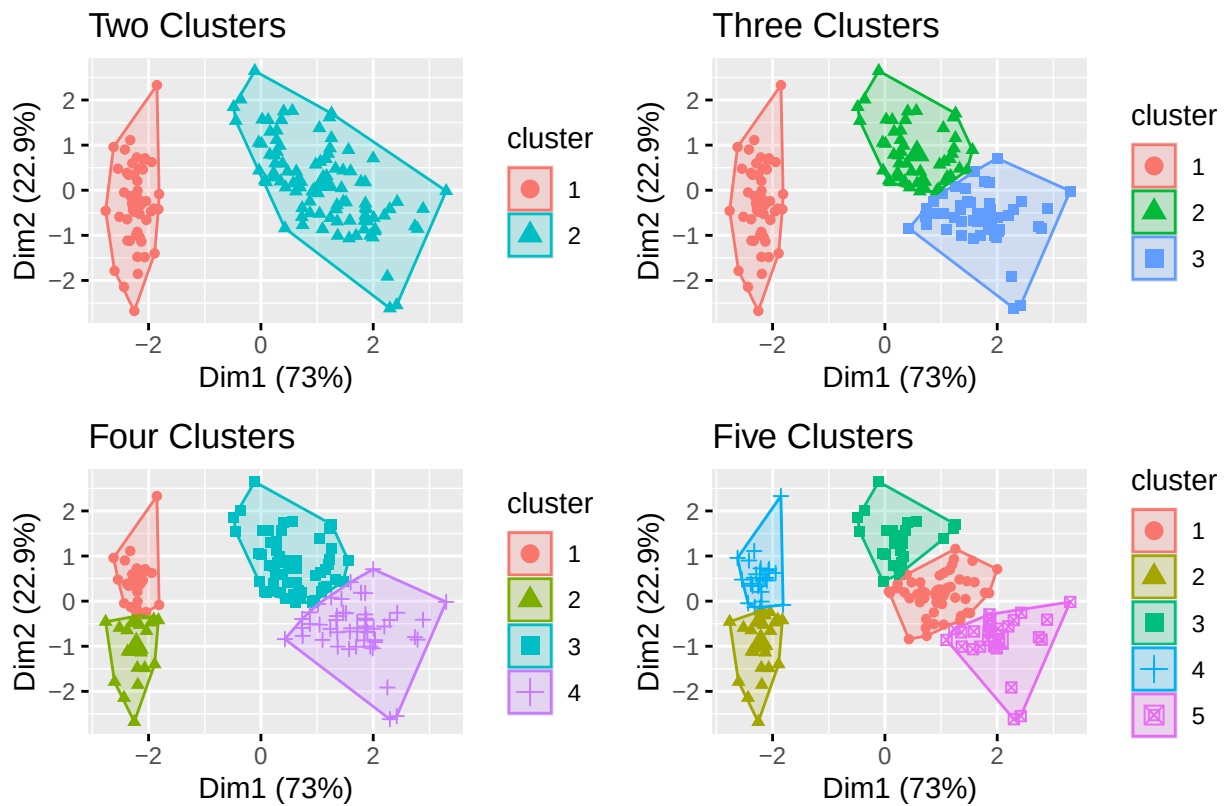
```
plot3 = fviz_cluster(km3, geom = "point", data = iris_clean) + ggtitle("Three Clusters")
```

```
plot4 = fviz_cluster(km4, geom = "point", data = iris_clean) + ggtitle("Four Clusters")
```

```
plot5 = fviz_cluster(km5, geom = "point", data = iris_clean) + ggtitle("Five Clusters")
```

```
grid.arrange(plot2, plot3, plot4, plot5, nrow = 2, top = "Figure 1: Cluster Plots")
```

Figure 1: Cluster Plots



```
table(km2$cluster, iris$Species)
```

```
##
##      setosa versicolor virginica
##  1       50           0          0
##  2        0          50         50
```

```
table(km3$cluster, iris$Species)
```

```
##
##      setosa versicolor virginica
##  1       50           0          0
##  2        0          39         14
##  3        0          11         36
```

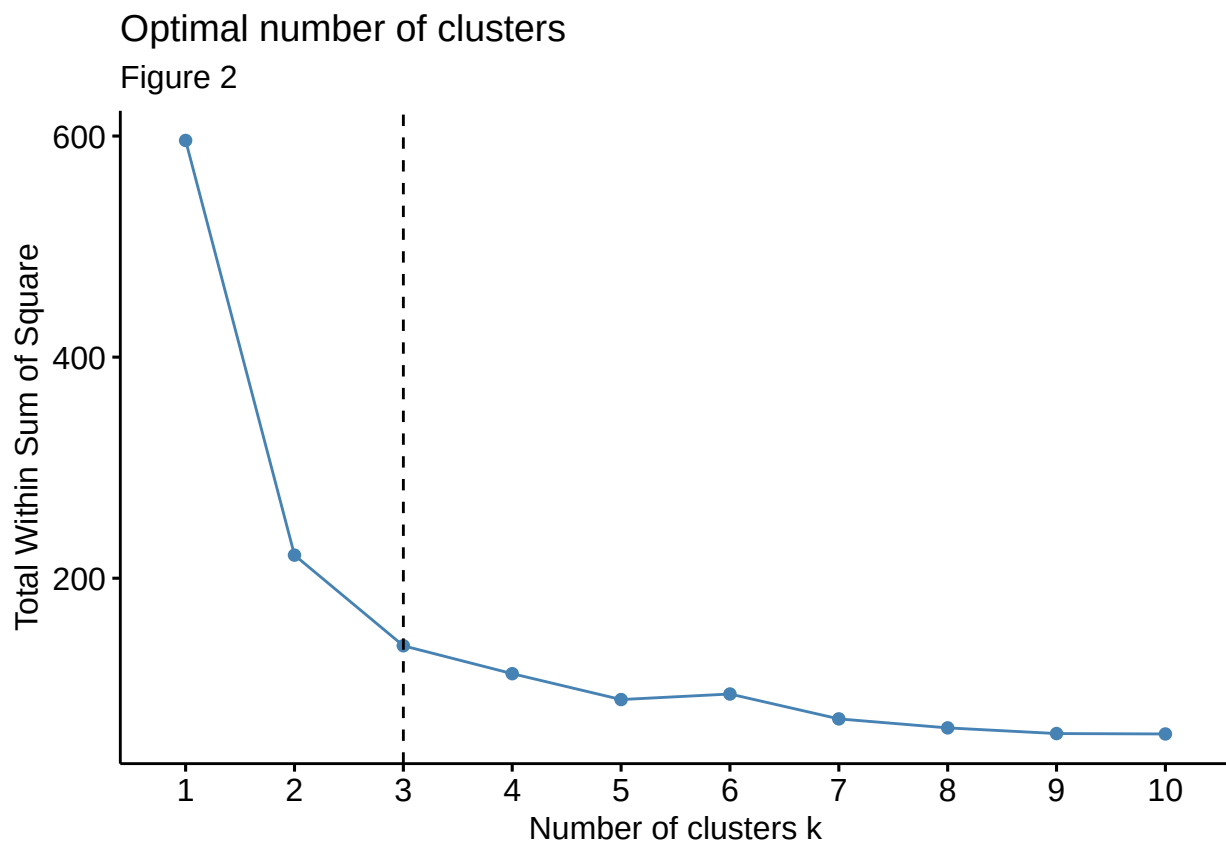
```
table(km4$cluster, iris$Species)
```

```
##
##      setosa versicolor virginica
##  1       25           0          0
##  2       25           0          0
##  3        0          39         14
##  4        0          11         36
```

```
table(km5$cluster, iris$Species)
```

```
##
##      setosa versicolor virginica
## 1         0          27         21
## 2        28           0           0
## 3         0          21           2
## 4        22           0           0
## 5         0           2          27
```

```
fviz_nbclust(iris_clean, kmeans, method = "wss") +
  geom_vline(xintercept = 3, linetype = 2) +
  labs(subtitle = "Figure 2")
```



```
accuracy_kmeans = list()
accuracyAll = (50+39+36) / 150
accuracy_kmeans[1] = accuracyAll
cat("Prediction Accuracy with all Predictors: ", as.integer(accuracyAll * 100), "% \n")
```

```
## Prediction Accuracy with all Predictors: 83 %
```

```
predictor_function = function(x1, x2, x3) {
  predictor_kmeans = kmeans(iris_clean[,c(x1, x2, x3)], 3, nstart = 20)
  table(predictor_kmeans$cluster, iris$Species)
}
```

```
predictor_function(1, 2, 3)
```

```
##  
##      setosa versicolor virginica  
##  1      49          0          0  
##  2       1         35         13  
##  3       0         15         37
```

```
predictor_function(1, 2, 4)
```

```
##  
##      setosa versicolor virginica  
##  1      49          0          0  
##  2       1         38         16  
##  3       0         12         34
```

```
predictor_function(2, 3, 4)
```

```
##  
##      setosa versicolor virginica  
##  1      49          0          0  
##  2       1         42         12  
##  3       0          8         38
```

```
predictor_function(1, 3, 4)
```

```
##  
##      setosa versicolor virginica  
##  1       0          5         36  
##  2       0         44         14  
##  3      50          1          0
```

```
accuracy1 = (49 + 35 + 37) / 150
```

```
accuracy2 = (49 + 38 + 34) / 150
```

```
accuracy3 = (49 + 42 + 38) / 150
```

```
accuracy4 = (50 + 44 + 36) / 150
```

```
c(accuracy1, accuracy2, accuracy3, accuracy4)
```

```
## [1] 0.8066667 0.8066667 0.8600000 0.8666667
```

The best combination is the fourth accuracy percentage, which values at 86.66667%. This would mean that the best combination of predictors is sepal length, petal length, and petal width.